



Nama/NIM : Claudya Destine Julia Handoko/2341760008
Mata Kuliah : Pemrograman Web Lanjut (PWL)
Program Studi : D4 – Teknik Informatika / D4 – Sistem Informasi Bisnis
Semester : 4 (empat) / 5 (lima)
Pertemuan ke- : 7 (tujuh)

JOBSHEET 07

Authentication dan Authorization di Laravel

Laravel Authentication dipergunakan untuk memproteksi halaman atau fitur dari web yang hanya diakses oleh orang tertentu yang diberikan hak. Fitur seperti ini biasanya ditemui di sistem yang memiliki fitur administrator atau sistem yang memiliki pengguna yang boleh menambahkan datanya.

Laravel membuat penerapan otentikasi sangat sederhana dan telah menyediakan berbagai fitur yang dapat dimanfaatkan tanpa perlu melakukan penambahan instalasi modul tertentu. File konfigurasi otentikasi terletak di `config / auth.php`, yang berisi beberapa opsi yang terdokumentasi dengan baik untuk mengubah konfigurasi dari layanan otentikasi.

Pada intinya, fasilitas otentikasi Laravel terdiri dari “*guards*” dan “*providers*”. *Guards* menentukan bagaimana pengguna diautentikasi untuk setiap permintaan. Misalnya, Laravel mengirim dengan *guards* untuk sesi dengan menggunakan penyimpanan session dan cookie.

Middleware

Middleware adalah lapisan perantara antara permintaan **route HTTP** yang masuk dan **action** dari Controller yang akan dijalankan. **Middleware** memungkinkan kita untuk melakukan berbagai tugas baik itu sebelum ataupun sesudah tindakan dilakukan. Kita juga dapat menggunakan **tool CLI** untuk membuat sebuah **Middleware** dalam **Laravel**. Beberapa contoh penggunaan **Middleware** meliputi autentikasi, validasi, manipulasi permintaan, dan lainnya. Berikut di bawah ini adalah manfaat dari **Middleware** :

- **Keamanan** : dalam **Middleware** memungkinkan kita untuk memverifikasi apakah pengguna sudah diautentikasi sebelum mengakses halaman tertentu. Dengan demikian, kita dapat melindungi data sensitif dan mengontrol hak akses pengguna.



- **Pemfilteran Data : Middleware** dapat digunakan untuk memanipulasi data permintaan sebelum sebuah **action** dalam **controller** dilakukan. Misalnya, kita dapat memeriksa terlebih dahulu data yang dikirim oleh pengguna sebelum data tersebut diproses lebih lanjut atau kita ingin memodifikasi data yang akan dikirim lalu kita dapat memeriksa ulang data yang akan dikirim oleh pengguna sebelum data tersebut diproses.
- **Logging dan Audit : Middleware** juga dapat digunakan untuk mencatat aktivitas pengguna atau melakukan audit terhadap permintaan yang masuk. Ini dapat membantu dalam pemantauan dan analisis aplikasi.

INFO

Kita akan menggunakan Laravel Auth secara manual seperti
<https://laravel.com/docs/10.x/authentication#authenticating-users>

Sesuai dengan **Studi Kasus PWL.pdf**.

Jadi project Laravel 10 kita masih sama dengan menggunakan repositori **PWL_POS**.

Project **PWL_POS** akan kita gunakan sampai pertemuan 12 nanti, sebagai project yang akan kita pelajari

A. Implementasi Manual Authentication di Laravel

Autentikasi adalah proses untuk memverifikasi identitas pengguna yang mencoba mengakses sistem. Dalam konteks aplikasi web, autentikasi memastikan bahwa pengguna yang mencoba login memiliki hak akses yang sesuai berdasarkan kredensial seperti email dan password. Proses autentikasi berbeda dengan **otorisasi**, yang merupakan langkah lanjutan untuk menentukan hak akses apa yang dimiliki pengguna setelah mereka berhasil diautentikasi.

Konsep Autentikasi di Laravel

Laravel menawarkan sistem autentikasi yang sangat fleksibel. Laravel menyediakan mekanisme autentikasi bawaan melalui layanan authentication scaffolding seperti Laravel *Jetstream* dan *Breeze*, yang dapat secara otomatis menghasilkan halaman dan logika autentikasi. Namun, terkadang pengembang memerlukan implementasi autentikasi yang lebih manual untuk memberikan kontrol penuh terhadap setiap aspek dari proses tersebut.



Beberapa komponen penting dalam sistem autentikasi Laravel meliputi:

- *Guard*: Komponen yang mengatur bagaimana pengguna diautentikasi untuk setiap permintaan. *Guard* default menggunakan sesi dan cookie.
- *Provider*: Mengatur bagaimana pengguna diambil dari database atau sumber data lainnya. *Provider* default mengambil data pengguna dari database dengan menggunakan Eloquent ORM.
- *Session*: Laravel menggunakan sesi untuk menyimpan status autentikasi pengguna. Sesi memungkinkan sistem untuk mengingat pengguna yang sudah login di antara permintaan HTTP yang berbeda.

Alur umum dari autentikasi meliputi:

1. *Login*: Pengguna mengirimkan kredensial (biasanya berupa email dan password).
2. *Verifikasi Kredensial*: Sistem memeriksa apakah kredensial yang diberikan sesuai dengan data di database.
3. *Pembuatan Sesi*: Jika kredensial benar, sistem akan membuat sesi untuk pengguna yang akan disimpan di server.
4. *Akses ke Halaman yang Dilindungi*: Pengguna yang terautentikasi dapat mengakses halaman-halaman yang dilindungi oleh *middleware* auth.
5. *Logout*: Pengguna bisa keluar dari sistem dan sesi mereka akan dihapus.

Middleware Autentikasi

Middleware auth di Laravel digunakan untuk melindungi rute atau halaman agar hanya dapat diakses oleh pengguna yang sudah terautentikasi. Jika pengguna mencoba mengakses rute yang memerlukan autentikasi tanpa login, mereka akan diarahkan ke halaman login.

- ***Guard*** bertanggung jawab untuk menangani proses autentikasi pengguna. Laravel secara default menggunakan *guard* berbasis sesi untuk autentikasi web, namun juga mendukung *guard* berbasis token (seperti API).
- ***Provider*** bertugas untuk mengambil pengguna dari database. Laravel menyediakan *provider* default yang menggunakan Eloquent, namun juga mendukung *provider* lain seperti Query Builder.



Implementasi di Laravel 10

Kita akan menerapkan menggunakan authentication di Laravel. Dalam penerapan ini, kita akan mencoba membuat otentikasi secara di Laravel, agar kita paham langkah-langkah dalam membuat Authentication

Praktikum 1 – Implementasi Authentication :

1. Kita buka project laravel **PWL_POS** kita, dan kita modifikasi konfigurasi aplikasi kita di `config/auth.php`

```
62     'providers' => [  
63         'users' => [  
64             'driver' => 'eloquent',  
65             'model' => App\Models\User::class,  
66         ],
```

Pada bagian ini kita sesuaikan dengan Model untuk tabel `m_user` yang sudah kita buat

```
62     'providers' => [  
63         'users' => [  
64             'driver' => 'eloquent',  
65             'model' => App\Models\UserModel::class,  
66         ],  
67     ],
```

Hasil :

```
config > auth.php  
3     return [  
45         /*  
60         */  
61         //js7  
62         'providers' => [  
63             'users' => [  
64                 'driver' => 'eloquent',  
65                 'model' => App\Models\UserModel::class,  
66             ],
```

2. Selanjutnya kita modifikasi sedikit pada `UserModel.php` untuk bisa melakukan proses otentikasi



```
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Foundation\Auth\User as Authenticatable; // implementasi class Authenticatable

class UserModel extends Authenticatable
{
    use HasFactory;

    protected $table = 'm_user';
    protected $primaryKey = 'user_id';
    protected $fillable = ['username', 'password', 'nama', 'level_id', 'created_at', 'updated_at'];

    protected $hidden = ['password']; // jangan di tampilkan saat select

    protected $casts = ['password' => 'hashed']; // casting password agar otomatis di hash

    /**
     * Relasi ke tabel level
     */
    public function level(): BelongsTo
    {
        return $this->belongsTo(LevelModel::class, 'level_id', 'level_id');
    }
}
```

Hasil :



```
app > Models > UserModel.php > PHP > UserModel
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6  use Illuminate\Database\Eloquent\Factories\HasFactory;
7  use Illuminate\Database\Eloquent\Relations\BelongsTo;
8  use Illuminate\Foundation\Auth\User as Authenticatable; // implementasi class Authenticatable
9
10 15 references | 0 implementations
11 class UserModel extends Authenticatable
12 {
13     use HasFactory;
14
15     2 references
16     protected $table = 'm_user';
17     2 references
18     protected $primaryKey = 'user_id';
19     4 references
20     protected $fillable = ['username', 'password', 'nama', 'level_id', 'created_at', 'updated_at'];
21     6 references
22     protected $hidden = ['password']; // jangan di tampilkan saat select
23     6 references
24     protected $casts = ['password' => 'hashed']; // casting password agar otomatis di hash
25
26     /**
27      * Relasi ke tabel level
28      */
29     0 references | 0 overrides
30     public function level(): BelongsTo
31     {
32         return $this->belongsTo(LevelModel::class, 'level_id', 'level_id');
33     }
34 }
```

3. Selanjutnya kita buat `AuthController.php` untuk memproses login yang akan kita lakukan



```
<?php
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;

class AuthController extends Controller
{
    public function login()
    {
        if(Auth::check()){ // jika sudah login, maka redirect ke halaman home
            return redirect('/');
        }
        return view('auth.login');
    }
    public function postlogin(Request $request)
    {
        if($request->ajax() || $request->wantsJson()){
            $credentials = $request->only('username', 'password');
            if (Auth::attempt($credentials))
            {
                return response()->json([
                    'status' => true,
                    'message' => 'Login Berhasil',
                    'redirect' => url('/')
                ]);
            }
            return response()->json([
                'status' => false,
                'message' => 'Login Gagal'
            ]);
        }

        return redirect('login');
    }
    public function logout(Request $request)
    {
        Auth::logout();

        $request->session()->invalidate();
        $request->session()->regenerateToken();
        return redirect('login');
    }
}
```

Hasil :



```
app > Http > Controllers > AuthController.php > PHP Intelephense > AuthController
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use Illuminate\Http\Request;
6  use Illuminate\Support\Facades\Auth;
7
8  0 references | 0 implementations
9  class AuthController extends Controller
10 {
11     0 references | 0 overrides
12     public function login(): Factory|Redirector|RedirectResponse|View
13     {
14         if (Auth::check()) { // jika sudah login, maka redirect ke halaman home
15             return redirect(to: '/');
16         }
17         return view(view: 'auth.login');
18     }
19
20     0 references | 0 overrides
21     public function postlogin(Request $request): JsonResponse|mixed|Redirector|RedirectRes...
22     {
23         if ($request->ajax() || $request->wantsJson()) {
24             $credentials = $request->only('username', 'password');
25             if (Auth::attempt($credentials)) {
26                 return response()->json([
27                     'status' => true,
28                     'message' => 'Login Berhasil',
29                     'redirect' => url(path: '/')
30                 ]);
31             }
32             return response()->json([
33                 'status' => false,
34                 'message' => 'Login Gagal'
35             ]);
36         }
37         return redirect(to: 'login');
38     }
39 }
```

4. Setelah kita membuat `AuthController.php`, kita buat view untuk menampilkan halaman login. View kita buat di `auth/login.blade.php`, tampilan login bisa kita ambil dari contoh login di template **AdminLTE** seperti berikut (pada contoh login ini, kita gunakan page `login-V2` di **AdminLTE**)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Login Pengguna</title>
```




```
<!-- Google Font: Source Sans Pro -->
<link rel="stylesheet"
href="https://fonts.googleapis.com/css?family=Source+Sans+Pro:300,400,400i,700&display=fallb
ack">
<!-- Font Awesome -->
<link rel="stylesheet" href="{{ asset('plugins/fontawesome-free/css/all.min.css') }}">
<!-- icheck bootstrap -->
<link rel="stylesheet" href="{{ asset('plugins/icheck-bootstrap/icheck-bootstrap.min.css')
}}">
<!-- SweetAlert2 -->
<link rel="stylesheet" href="{{ asset('plugins/sweetalert2-theme-bootstrap-
4/bootstrap4.min.css') }}">
<!-- Theme style -->
<link rel="stylesheet" href="{{ asset('dist/css/adminlte.min.css') }}"> </head>
<body class="hold-transition login-page">
<div class="login-box">
  <!-- /.login-logo -->
  <div class="card card-outline card-primary">
    <div class="card-header text-center"><a href="{{ url('/') }}"
class="h1"><b>Admin</b>LTE</a></div>
    <div class="card-body">
      <p class="login-box-msg">Sign in to start your session</p>
      <form action="{{ url('login') }}" method="POST" id="form-login">
@csrf
        <div class="input-group mb-3">
          <input type="text" id="username" name="username" class="form-control"
placeholder="Username">
          <div class="input-group-append">
            <div class="input-group-text">
              <span class="fas fa-envelope"></span>
            </div>
          </div>
          <small id="error-username" class="error-text text-danger"></small>
        </div>
        <div class="input-group mb-3">
          <input type="password" id="password" name="password" class="form-control"
placeholder="Password">
          <div class="input-group-append">
            <div class="input-group-text">
              <span class="fas fa-lock"></span>
            </div>
          </div>
          <small id="error-password" class="error-text text-danger"></small>
        </div>
        <div class="row">
          <div class="col-8">
            <div class="icheck-primary">
              <input type="checkbox" id="remember"><label for="remember">Remember Me</label>
            </div>
          <!-- /.col -->
          <div class="col-4">
            <button type="submit" class="btn btn-primary btn-block">Sign In</button>
          </div>
          <!-- /.col -->
        </div>
      </form>
    </div>
    <!-- /.card-body -->
  </div>
```



KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
<http://www.polinema.ac.id>

```
<!-- /.card -->  
</div>  
<!-- /.login-box -->  
  
<!-- jQuery -->
```



```
<script src="{{ asset('plugins/jquery/jquery.min.js') }}"></script>
<!-- Bootstrap 4 -->
<script src="{{ asset('plugins/bootstrap/js/bootstrap.bundle.min.js') }}"></script> <!--
jquery-validation -->
<script src="{{ asset('plugins/jquery-validation/jquery.validate.min.js') }}"></script>
<script src="{{ asset('plugins/jquery-validation/additional-methods.min.js') }}"></script>
<!-- SweetAlert2 -->
<script src="{{ asset('plugins/sweetalert2/sweetalert2.min.js') }}"></script> <!--
AdminLTE App -->
<script src="{{ asset('dist/js/adminlte.min.js') }}"></script>
<script>
$.ajaxSetup({    head
ers: {
    'X-CSRF-TOKEN': $('meta[name="csrf-token"]').attr('content')
    }
});

$(document).ready(function() {    $("#form-
login").validate({        rules: {            username: {required:
true, minlength: 4, maxlength: 20},            password: {required:
true, minlength: 6, maxlength: 20}
        },
        submitHandler: function(form) { // ketika valid, maka bagian yg akan dijalankan
            $.ajax({                url:
form.action,                type: form.method,
data:                $(form).serialize(),
success: function(response) {
                if(response.status){ // jika sukses
Swal.fire({                    icon: 'success',
title: 'Berhasil',                    text:
response.message,                }).then(function() {
                    window.location = response.redirect;
                });
            }else{ // jika error
                $('.error-text').text('');
                $.each(response.msgField, function(prefix, val) {
                    $('#error-
'+prefix).text(val[0]);                });
Swal.fire({                    ic
on: 'error',
                title: 'Terjadi Kesalahan',
text: response.message
                });
            }
        });
        return false;
    },
    errorElement: 'span',
    errorPlacement: function (error, element) {        error.addClass('invalid-
feedback');
        element.closest('.input-group').append(error);
    },
    highlight: function (element, errorClass, validClass) {
        $(element).addClass('is-invalid');
    },
    unhighlight: function (element, errorClass, validClass)
    {
        $(element).removeClass('is-invalid');
    }
});
```



```
});  
</script>
```



KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
<http://www.polinema.ac.id>

</body>
</html>

Hasil :



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="utf-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>Login Pengguna</title>
7   <!-- Google Fonts Source Sans Pro -->
8   <link rel="stylesheet"
9     href="https://fonts.googleapis.com/css?family=Source+Sans+Pro:300,400,400i,700&idisplay=fall&idisplay=auto">
10   <!-- Font Awesome -->
11   <link rel="stylesheet" href="{ asset('adminlte/plugins/fontawesome-free/css/all.min.css') }">
12   <!-- iCheck bootstrap -->
13   <link rel="stylesheet" href="{ asset('adminlte/plugins/iCheck-bootstrap/iCheck-bootstrap.min.css') }">
14   <!-- SweetAlert2 -->
15   <link rel="stylesheet" href="{ asset('adminlte/plugins/sweetalert2-theme-bootstrap-4/bootstrap.min.css') }">
16   <!-- Theme style -->
17   <link rel="stylesheet" href="{ asset('adminlte/dist/css/adminlte.min.css') }">
18 </head>
19 <body class="hold-transition login-page">
20 <div class="login-box">
21   <!-- /.login-page -->
22   <div class="card card-outline card-primary">
23     <div class="card-header text-center"><a href="{ url('/') }" class="h1">Admin</a></div>
24     <div class="card-body">
25       <p class="login-box-msg">Sign in to start your session</p>
26       <form action="{ url('login') }" method="post" id="form-login">
27         <@csrf>
28         <div class="input-group mb-3">
29           <input type="text" id="username" name="username" class="form-control" placeholder="Username">
30           <div class="input-group-append">
31             <div class="input-group-text">
32               <span class="fas fa-envelope"></span>
33             </div>
34           </div>
35         </div>
36         <small id="error-username" class="error-text text-danger"></small>
37         <div class="input-group mb-3">
38           <input type="password" id="password" name="password" class="form-control" placeholder="Password">
39           <div class="input-group-append">
40             <div class="input-group-text">
41               <span class="fas fa-lock"></span>
42             </div>
43           </div>
44         </div>
45         <small id="error-password" class="error-text text-danger"></small>
46         <div class="row">
47           <div class="col-8">
48             <div class="checkbox">
49               <input type="checkbox" id="remember"><label for="remember">Remember Me</label>
50             </div>
51           </div>
52           <div class="col-4">
53             <button type="submit" class="btn btn-primary btn-block">Sign In</button>
54           </div>
55         </div>
56         <!-- /.col -->
57       </div>
58     </form>
59   </div>
60   <!-- /.card-body -->
61 </div>
62 <!-- /.card -->
63 </div>
64 <!-- /.login-box -->
65
66 <!-- jQuery -->
67 <script src="{ asset('plugins/jquery/jquery.min.js') }"></script>
68 <!-- Bootstrap 4 -->
69 <script src="{ asset('plugins/bootstrap/js/bootstrap.bundle.min.js') }"></script>
70 <!-- jQuery validation -->
71 <script src="{ asset('plugins/jquery-validation/jquery.validate.min.js') }"></script>
72 <script src="{ asset('plugins/jquery-validation/additional-methods.min.js') }"></script>
73 <!-- SweetAlert2 -->
74 <script src="{ asset('plugins/sweetalert2/sweetalert2.min.js') }"></script>
75 <!-- AdminLTE App -->
76 <script src="{ asset('dist/js/adminlte.min.js') }"></script>
77 </script>
78
79 $.ajaxSetup({
80   headers: {
81     'X-CSRF-TOKEN': $('meta[name="csrf-token"]').attr('content')
82   }
83 });
84
85 $(document).ready(function() {
86   $('#form-login').validate({
87     rules: {
88       username: {required: true, minlength: 4, maxlength: 20},
89       password: {required: true, minlength: 6, maxlength: 20}
90     },
91     submitHandler: function(form) { // ketika valid, maka bagian yg akan dijalankan
92       $.ajax({
93         url: form.action,
94         type: form.method,
95         data: $(form).serialize(),
96         success: function(response) {
97           if(response.status){ // jika sukses
98             Swal.fire({
99               icon: 'success',
100               title: 'Berhasil!',
101               text: response.message,
102             }).then(function() {
103               window.location = response.redirect;
104             });
105           } else { // jika error
106             $('#error-text').text('');
107             $.each(response.msgField, function(prefix, val) {
108               $('#error-' + prefix).text(val[0]);
109             });
110             Swal.fire({
111               icon: 'error',
112               title: 'Terjadi Kesalahan',
113               text: response.message,
114             });
115           }
116         }
117       });
118     }
119   });
120 });
121
122 // Highlight
123 highlight: function (element, errorClass, validClass) {
124   $(element).addClass('is-invalid');
125 },
126 unhighlight: function (element, errorClass, validClass) {
127   $(element).removeClass('is-invalid');
128 }
129
130 });
131 </script>
132 </body>
133 </html>
134 </head>
135 </html>
```



5. Kemudian kita modifikasi `route/web.php` agar semua route masuk dalam auth

```
<?php

use App\Http\Controllers\UserController;
use App\Http\Controllers\SupplierController;
use App\Http\Controllers\BarangController;
use App\Http\Controllers\AuthController;
use App\Http\Controllers\KategoriController;
use App\Http\Controllers\LevelController;
use App\Http\Controllers>WelcomeController;
use Illuminate\Support\Facades\Route;

Route::pattern('id', '[0-9]+'); // artinya ketika ada parameter {id}, maka harus berupa angka

Route::get('login', [AuthController::class, 'login'])->name('login');
Route::post('login', [AuthController::class, 'postlogin']);
Route::get('logout', [AuthController::class, 'logout'])->middleware('auth');

Route::middleware(['auth'])->group(function() { // artinya semua route di dalam group ini harus login dulu
    // masukkan semua route yang perlu autentikasi di sini
});
```

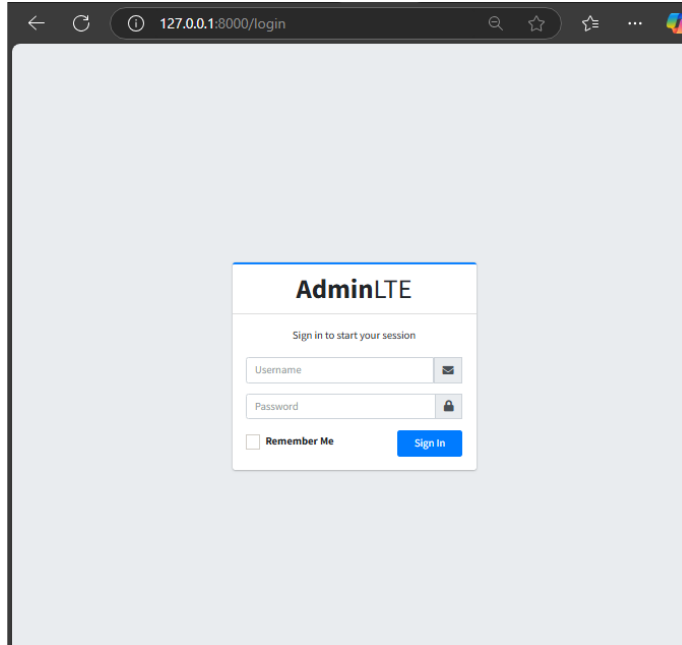
Hasil :

```
12 Route::pattern('id', '[0-9]+'); // artinya ketika ada parameter {id}, maka harus berupa angka
13
14 Route::get('login', [AuthController::class, 'login'])->name('login');
15 Route::post('login', [AuthController::class, 'postlogin']);
16 Route::get('logout', [AuthController::class, 'logout'])->middleware('auth');
17
18
19 Route::middleware(['auth'])->group(function () { void { // artinya semua route di dalam group ini harus login dulu
20     Route::get('/', [WelcomeController::class, 'index']);
21 }
```

6. Ketika kita coba mengakses halaman `localhost/PWL_POS/public` maka akan tampil halaman awal untuk login ke aplikasi



Hasil :

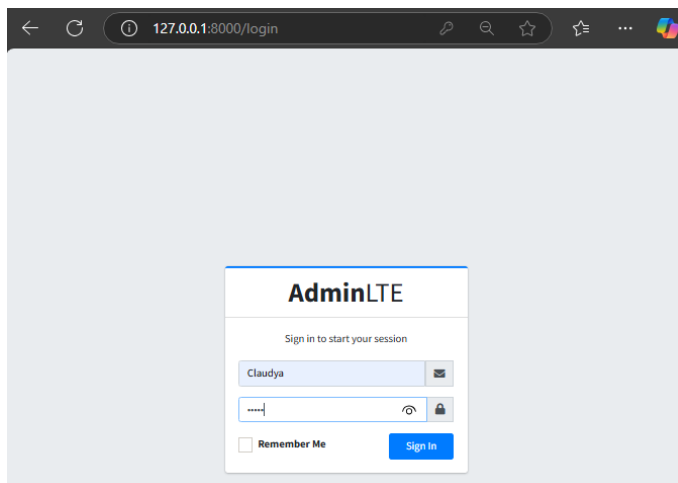


Tugas 1 – Implementasi Authentication :

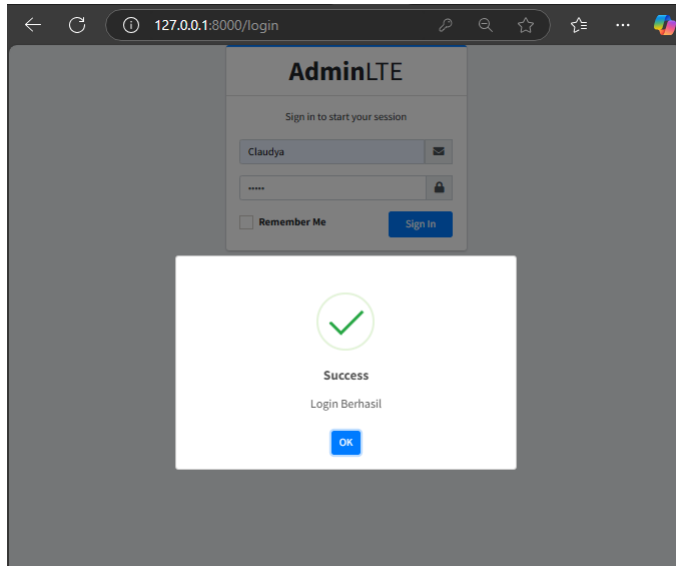
1. Silahkan implementasikan proses login pada project kalian masing-masing

Hasil :

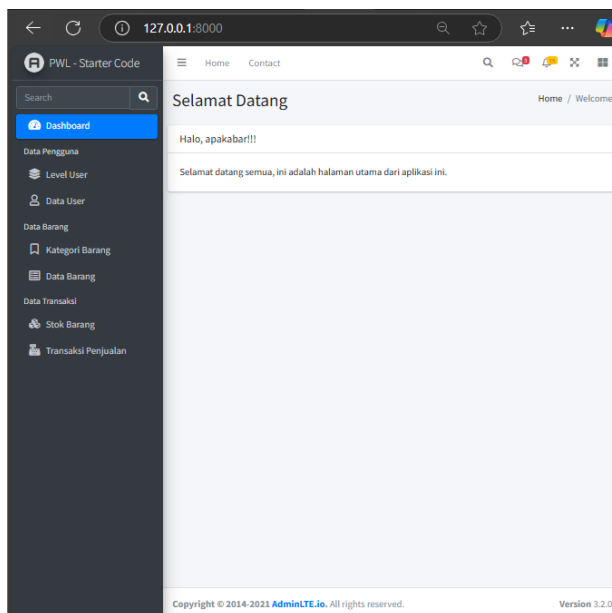
- Tampilan awal login



- Setelah berhasil melakukan login



- Masuk ke halaman dashboard



2. Silahkan implementasi proses logout pada halaman web yang kalian buat

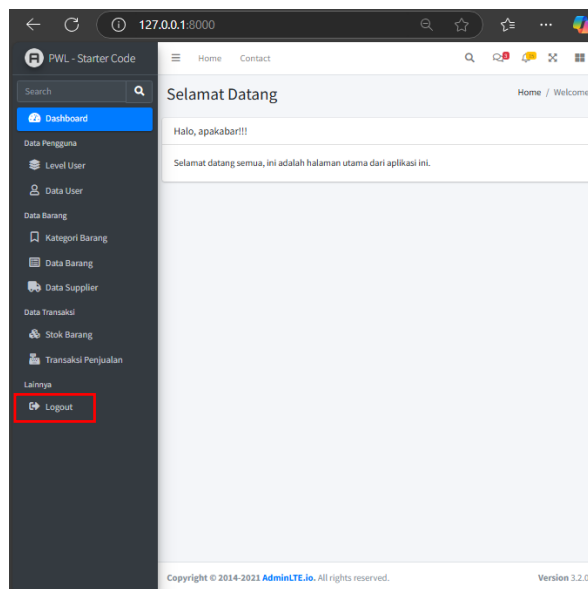
Hasil :

- Modifikasi bagian sidebar

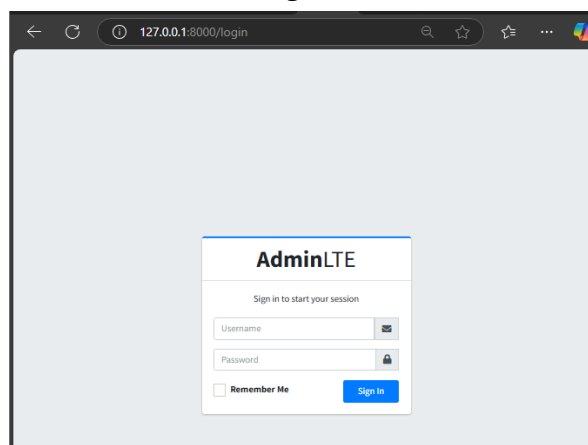


```
resources > views > layouts > sidebar.blade.php > ...
1  <div class="sidebar">
15  <nav class="mt-2">
16  <ul class="nav nav-pills nav-sidebar flex-column" data-widget="treeview"
70  <li class="nav-item">
71  <a href="{{ url(path: '/barang') }}" class="nav-link {{ ($activeMenu ==
75  </a>
76  </li>
77  <li class="nav-header">Lainnya</li>
78  <li class="nav-item">
79  <a href="{{ url(path: '/logout') }}" class="nav-link {{ ($activeMenu == 'logout')}
'active' : '' }}">
80  <i class="nav-icon fas fa-sign-out-alt"></i>
81  <p>Logout</p>
82  </a>
83  </li>
84  </ul>
85  </nav>
86  </div>
87
```

- Tampilan setelah modifikasi



- Setelah melakukan logout



3. Amati dan jelaskan tiap tahapan yang kalian kerjakan, dan jabarkan dalam laporan
4. Submit kode untuk impementasi Authentication pada repository github kalian.



B. Implementasi Authorization di Laravel

Authorization merupakan proses setelah *authentication* berhasil dilakukan (dalam kata lain, kita berhasil login ke sistem). *Authorization* berkenaan dengan hak akses pengguna dalam menggunakan sistem. *Authorization* memberikan/memastikan hak akses (ijin akses) kita, sesuai dengan aturan (role) yang ada di sistem. *Authorization* sangat penting untuk membatasi akses pengguna sesuai dengan peruntukannya.

Contoh ketika kita mengakses LMS dengan akun (*username* dan *password*) yang bertipe **Mahasiswa**. Saat berhasil melakukan *authentication*, maka hak akses kita juga akan diberikan selayaknya mahasiswa. Seperti melihat kursus (course), melihat materi, men-download file materi, mengerjakan/meng-upload tugas, mengikuti ujian, dll. Kita tidak akan diberikan hak akses oleh sistem untuk membuat materi, membuat soal ujian, membuat tugas, memberikan nilai tugas karena hak akses tersebut masuk ke ranah akun tipe **Dosen/Pengajar**.

Selain menyediakan layanan otentikasi bawaan, Laravel juga menyediakan cara sederhana untuk mengotorisasi tindakan pengguna terhadap sumber daya tertentu. Misalnya, meskipun pengguna diautentikasi, mereka mungkin tidak berwenang untuk memperbarui atau menghapus model Eloquent atau rekaman database tertentu yang dikelola oleh aplikasi Anda. Fitur otorisasi Laravel menyediakan cara yang mudah dan terorganisir untuk mengelola jenis pemeriksaan otorisasi ini.

Praktikum 2 – Implementasi Authorization di Laravel dengan Middleware

Kita akan menerapkan *authorization* pada project Laravel dengan menggunakan Middleware sebagai pengecekan akses. Langkah-langkah yang kita kerjakan sebagai berikut:

1. Kita modifikasi **UserModel.php** dengan menambahkan kode berikut



```
/**
 * Relasi ke tabel level
 */
public function level(): BelongsTo
{
    return $this->belongsTo(LevelModel::class, 'level_id', 'level_id');
}

/**
 * Mendapatkan nama role
 */
public function getRoleName(): string
{
    return $this->level->level_nama;
}

/**
 * Cek apakah user memiliki role tertentu
 */
public function hasRole($role): bool
{
    return $this->level->level_kode == $role;
}
```

Hasil :

```
app > Models > UserModel.php > PHP > UserModel
10 class UserModel extends Authenticatable
    6 references
    protected $casts = ['password' => 'hashed']; // casting password agar otomatis di hash
20
21
22 /**
23  * Relasi ke tabel level
24  */
    0 references | 0 overrides
25 public function level(): BelongsTo
26 {
27     return $this->belongsTo(LevelModel::class, 'level_id', 'level_id');
28 }
29
30 /**
31  * Mendapatkan nama role
32  */
    0 references | 0 overrides
33 public function getRoleName(): string
34 {
35     return $this->level->level_nama;
36 }
37
38 /**
39  * Cek apakah user memiliki role tertentu
40  */
    0 references | 0 overrides
41 public function hasRole($role): bool
42 {
43     return $this->level->level_kode == $role;
44 }
```



2. Kemudian kita buat *middleware* dengan nama `AuthorizeUser.php`. Kita bisa buat *middleware* dengan mengetikkan perintah pada terminal/CMD

```
php artisan make:middleware AuthorizeUser
```

File *middleware* akan dibuat di `app/Http/Middleware/AuthorizeUser.php`

Hasil :

```
PS C:\laragon\www\PWL2025\Week7\Jobsheet7> php artisan make:middleware AuthorizeUser
INFO  Middleware [C:\laragon\www\PWL2025\Week7\Jobsheet7\app\Http\Middleware\AuthorizeUser.php] created successfully.
```

3. Kemudian kita edit *middleware* `AuthorizeUser.php` untuk bisa mengecek apakah pengguna yang mengakses memiliki Level/Role/Group yang sesuai

```
1  <?php
2  namespace App\Http\Middleware;
3
4  use Closure;
5  use Illuminate\Http\Request;
6  use Symfony\Component\HttpFoundation\Response;
7
8  class AuthorizeUser
9  {
10     /**
11      * Handle an incoming request.
12      *
13      * @param  \Closure(\Illuminate\Http\Request): (\Symfony\Component\HttpFoundation\Response)
14      */
15     public function handle(Request $request, Closure $next, $role = ''): Response
16     {
17         $user = $request->user(); // ambil data user yg login
18         // fungsi user() diambil dari UserModel.php
19         if($user->hasRole($role)){ // cek apakah user punya role yg diinginkan
20             return $next($request);
21         }
22         // jika tidak punya role, maka tampilkan error 403
23         abort(403, 'Forbidden. Kamu tidak punya akses ke halaman ini');
24     }
25 }
```

Hasil :



```
app > Http > Middleware > AuthorizeUser.php > PHP Intelephense > AuthorizeUser
1  <?php
2
3  namespace App\Http\Middleware;
4
5  use Closure;
6  use Illuminate\Http\Request;
7  use Symfony\Component\HttpFoundation\Response;
8
9  class AuthorizeUser
10
11     /**
12      * Handle an incoming request.
13      *
14      * @param  \Closure(\Illuminate\Http\Request): (\Symfony\Component\HttpFoundation\Response)  $next
15      */
16     public function handle(Request $request, Closure $next, $role): Response
17     {
18         $user = $request->user(); // ambil data user yg login
19         // fungsi user() diambil dari UserModel.php
20         if ($user->hasRole($role)) { // cek apakah user punya role yg diinginkan
21             return $next($request);
22         }
23
24         // jika tidak punya role, maka tampilkan error 403
25         abort(code: 403, message: 'Forbidden. Kamu tidak punya akses ke halaman ini');
26     }
27 }
```

4. Kita daftarkan ke `app/Http/Kernel.php` untuk *middleware* yang kita buat barusan

```
protected $middlewareAliases = [
    'auth' => \App\Http\Middleware\Authenticate::class,
    'authorize' => \App\Http\Middleware\AuthorizeUser::class, // middleware yg kita buat
    'auth.basic' => \Illuminate\Auth\Middleware\AuthenticateWithBasicAuth::class,
    'auth.session' => \Illuminate\Session\Middleware\AuthenticateSession::class,
```

Hasil :

```
app > Http > Kernel.php > PHP > Kernel > middlewareAliases
7  class Kernel extends HttpKernel
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55  protected $middlewareAliases = [
56      'auth' => \App\Http\Middleware\Authenticate::class,
57      'authorize' => \App\Http\Middleware\AuthorizeUser::class, // middleware yg kita buat
58      'auth.basic' => \Illuminate\Auth\Middleware\AuthenticateWithBasicAuth::class,
59      'auth.session' => \Illuminate\Session\Middleware\AuthenticateSession::class,
60      'cache.headers' => \Illuminate\Http\Middleware\SetCacheHeaders::class,
```

5. Sekarang kita perhatikan tabel `m_level` yang menjadi tabel untuk menyimpan level/group/role dari user ada

level_id	level_kode	level_nama	created_at	updated_at	deleted_at
1	ADM	Administrator	NULL	NULL	NULL
2	MNG	Manager	NULL	NULL	NULL
3	STF	Staf	NULL	2024-08-16 01:49:20	NULL
4	KSR	Kasir	NULL	NULL	NULL

Hasil :



	level_id	level_kode	level_nama	created_at	updated_at
☐ Edit ☐ Copy ☐ Delete	1	ADM	Administrator	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	2	MNG	Manager	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	3	STF	Staff/Kasir	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	4	CUS2	Customer	2025-03-05 15:25:22	NULL

6. Untuk mencoba *authorization* yang telah kita buat, maka perlu kita modifikasi `route/web.php` untuk menentukan route mana saja yang akan diberi hak akses sesuai dengan level user

```
Route::middleware(['auth'])->group(function(){ // artinya semua route di dalam group ini harus login dulu
    Route::get('/', [WelcomeController::class,'index']);
    // route Level

    // artinya semua route di dalam group ini harus punya role ADM (Administrator)
    Route::middleware(['authorize:ADM'])->group(function(){
        Route::get('/level', [LevelController::class,'index']);
        Route::post('/level/list', [LevelController::class,'list']); // untuk list json datatables
        Route::get('/level/create', [LevelController::class,'create']);
        Route::post('/level', [LevelController::class,'store']);
        Route::get('/level/{id}/edit', [LevelController::class,'edit']); // untuk tampilkan form edit
        Route::put('/level/{id}', [LevelController::class,'update']); // untuk proses update data
        Route::delete('/level/{id}', [LevelController::class,'destroy']); // untuk proses hapus data
    });
    // route Kategori
```

Hasil :

```
19 Route::middleware(['auth'])->group(function () { void // artinya semua route di dalam group ini harus login dulu
20 Route::get('/', [WelcomeController::class, 'index']); //route level
21
22 // artinya semua route di dalam group ini harus punya role ADM (Administrator)
23 Route::middleware(['authorize:ADM'])->group(function(): void {
24 Route::get('/level', [LevelController::class, 'index']);
25 Route::post('/level/list', [LevelController::class, 'list']); // untuk list json datatables
26 Route::get('/level/create', [LevelController::class, 'create']);
27 Route::post('/level', [LevelController::class, 'store']);
28 Route::get('/level/{id}/edit', [LevelController::class, 'edit']); // untuk tampilkan form edit
29 Route::put('/level/{id}', [LevelController::class, 'update']); // untuk proses update data
30 Route::delete('/level/{id}', [LevelController::class, 'destroy']); // untuk proses hapus data
31
32 });
33
34 // route Kategori
```

Pada kode yang ditandai merah, terdapat `authorize:ADM`. Kode `ADM` adalah nilai dari `level_kode` pada tabel `m_level`. Yang artinya, user yang bisa mengakses route untuk manage data level, adalah user yang memiliki level sebagai Administrator.

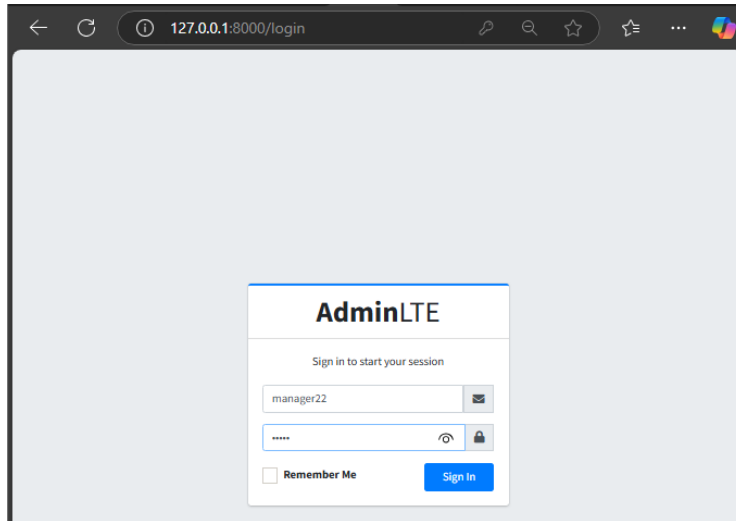
7. Untuk membuktikannya, sekarang kita coba login menggunakan akun selain level administrator, dan kita akses route menu level tersebut

Hasil :

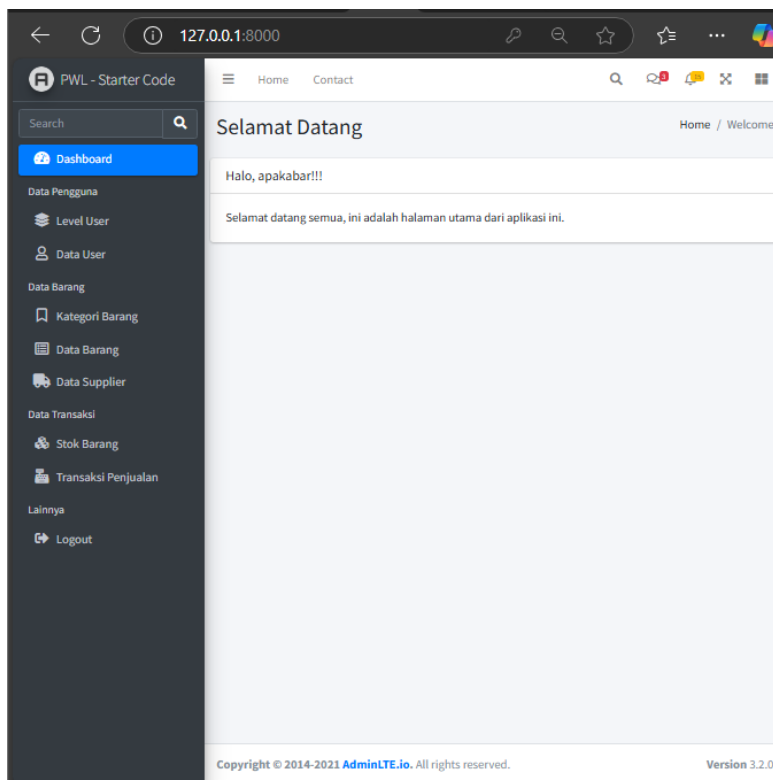
- Melakukan login selain role admin



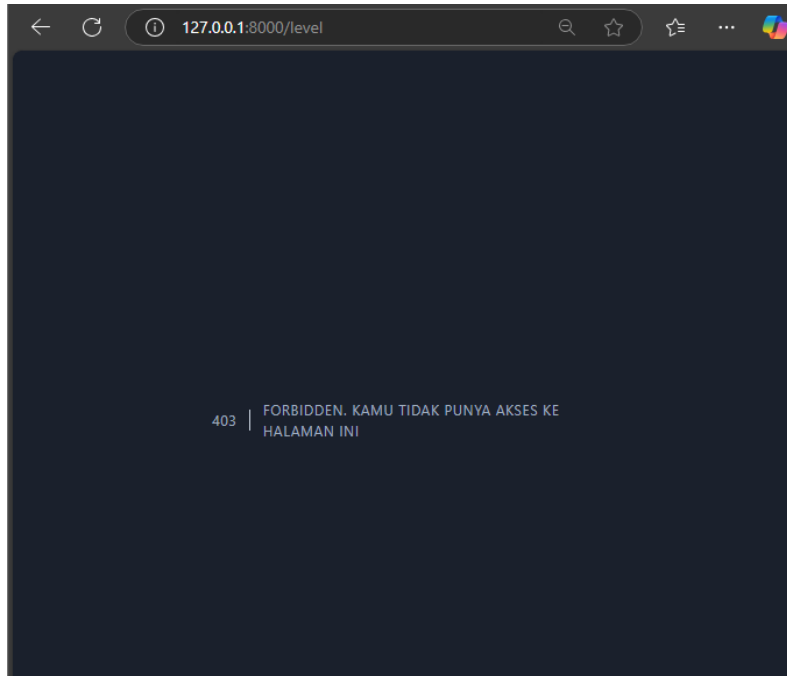
KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
<http://www.polinema.ac.id>



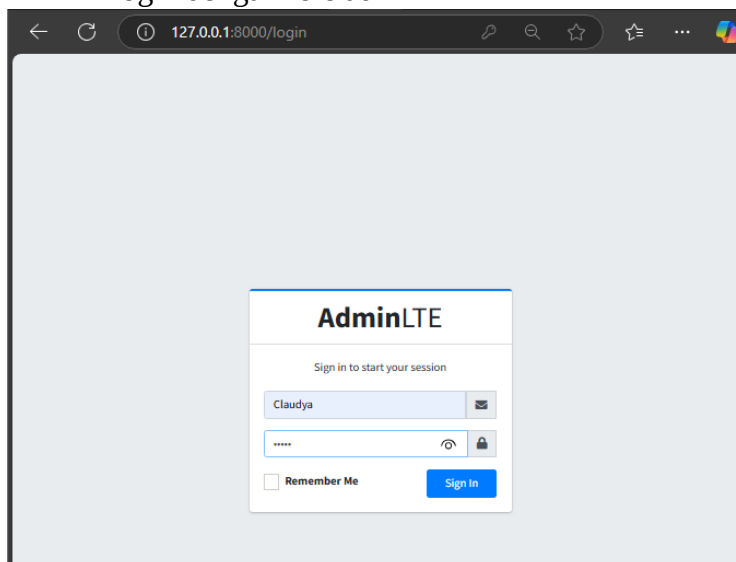
- Masuk ke halaman dashboard



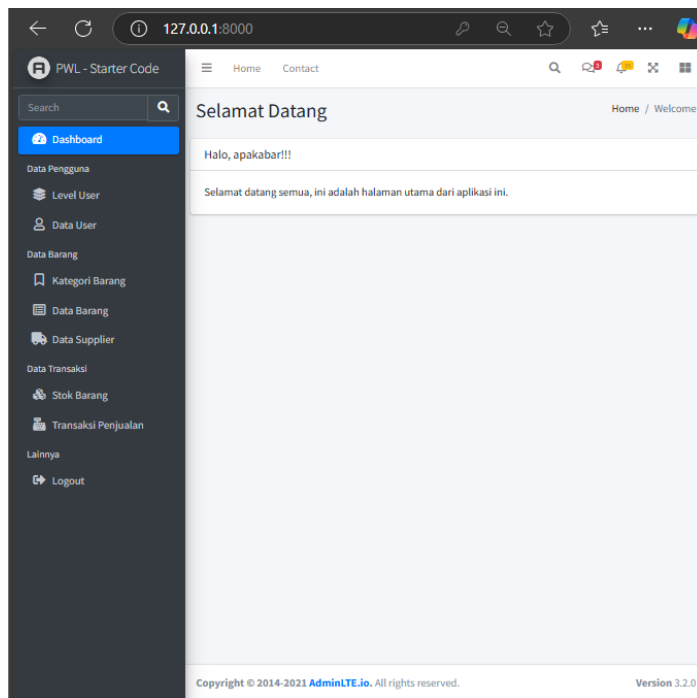
- Hasil jika login dengan role selain admin, ketika ingin mengakses level user akan menghasilkan seperti digambar karena izin akses hanya untuk role admin



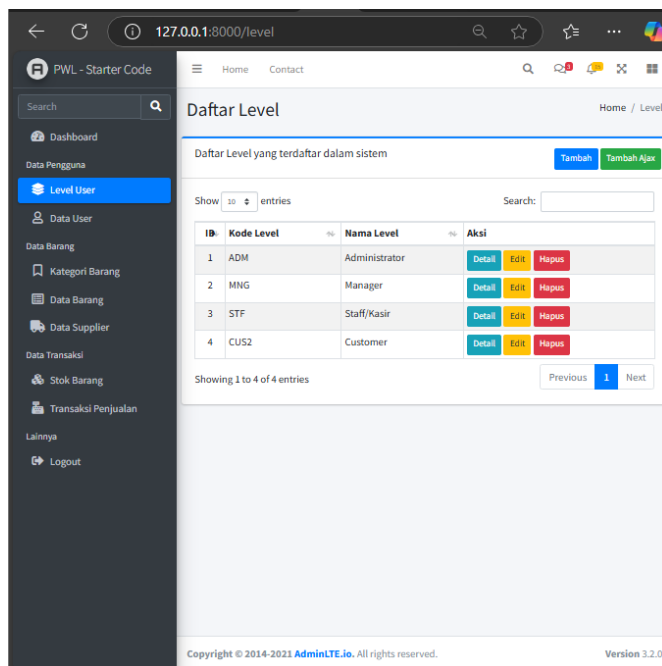
- Login dengan role admin



- Masuk halaman dashboard



- Hasil jika login dengan role admin, ketika mengakses level user akan menampilkan seperti digambar





Tugas 2 – Implementasi Authoriization :

1. Apa yang kalian pahami pada praktikum 2 ini?

Jawaban : Dari yang saya pahami, dengan menambahkan authorize:ADM pada suatu route, maka route tersebut hanya mengizinkan / bisa diakses oleh role yang di authorize yaitu ADM, sehingga role selain ADM tidak memiliki akses ke route tersebut dan akan menampilkan halaman eror 404

2. Amati dan jelaskan tiap tahapan yang kalian kerjakan, dan jabarkan dalam laporan
3. Submit kode untuk impementasi Authorization pada repository github kalian.

C. Multi-Level Authorization di Laravel

Bagaimana seandainya jika terdapat level/group/role satu dengan yang lain memiliki hak akses yang sama. Contoh sederhana, user level Admin dan Manager bisa sama-sama mengakses menu Barang pada aplikasi yang kita buat. Maka tidak mungkin kalau kita buat route untuk masing-masing level user. Hal ini akan memakan banyak waktu, dan proses yang lama.

```
// artinya semua route di dalam group ini harus punya role ADM (Administrator)
Route::middleware(['authorize:ADM'])->group(function(){
    Route::get('/barang',[BarangController::class,'index']);
    Route::post('/barang/list',[BarangController::class,'list']);
    Route::get('/barang/create_ajax',[BarangController::class,'create_ajax']); // ajax form create
    Route::post('/barang_ajax',[BarangController::class,'store_ajax']); // ajax store
    Route::get('/barang/{id}/edit_ajax',[BarangController::class,'edit_ajax']); // ajax form edit
    Route::put('/barang/{id}/update_ajax',[BarangController::class,'update_ajax']); // ajax update
    Route::get('/barang/{id}/delete_ajax',[BarangController::class,'confirm_ajax']); // ajax form confirm delete
    Route::delete('/barang/{id}/delete_ajax',[BarangController::class,'delete_ajax']); // ajax delete
});

// artinya semua route di dalam group ini harus punya role MNG (Manager)
Route::middleware(['authorize:MNG'])->group(function(){
    Route::get('/barang',[BarangController::class,'index']);
    Route::post('/barang/list',[BarangController::class,'list']);
    Route::get('/barang/create_ajax',[BarangController::class,'create_ajax']); // ajax form create
    Route::post('/barang_ajax',[BarangController::class,'store_ajax']); // ajax store
    Route::get('/barang/{id}/edit_ajax',[BarangController::class,'edit_ajax']); // ajax form edit
    Route::put('/barang/{id}/update_ajax',[BarangController::class,'update_ajax']); // ajax update
    Route::get('/barang/{id}/delete_ajax',[BarangController::class,'confirm_ajax']); // ajax form confirm delete
    Route::delete('/barang/{id}/delete_ajax',[BarangController::class,'delete_ajax']); // ajax delete
});
```

Hal ini jadi kendala ketika kita mau mengganti hak akses, maka kita akan mengganti sebagian besar route yang sudah kita tulis. Untuk itu, kita perlu mengelola middleware agar bisa mendukung penambahan hak akses secara dinamis.



Praktikum 3 – Implementasi Multi-Level Authorizaton di Laravel dengan Middleware

Kita akan menerapkan multi-level authorization pada project Laravel dengan menggunakan Middleware sebagai pengecekan akses. Langkah-langkah yang kita kerjakan sebagai berikut:

1. Kita modifikasi `UserModel.php` untuk mendapatkan `level_kode` dari user yang sudah login. Jadi kita buat fungsi dengan nama `getRole()`

```
/**
 * Mendapatkan nama role
 */
public function getRoleName(): string
{
    return $this->level->level_nama;
}

/**
 * Cek apakah user memiliki role tertentu
 */
public function hasRole($role): bool
{
    return $this->level->level_kode == $role;
}

/**
 * Mendapatkan kode role
 */
public function getRole()
{
    return $this->level->level_kode;
}
```

Hasil :

```
app > Models > UserModel.php > PHP > UserModel
10 class UserModel extends Authenticatable
11 {
12     //
13     //
14     //
15     //
16     //
17     //
18     /**
19      * Cek apakah user memiliki role tertentu
20      */
21     0 references | 0 overrides
22     public function hasRole($role): bool
23     {
24         return $this->level->level_kode == $role;
25     }
26     //
27     /**
28      * Mendapatkan kode role
29      */
30     0 references | 0 overrides
31     public function getRole(): mixed
32     {
33         return $this->level->level_kode;
34     }
35 }
```



2. Selanjutnya, Kita modifikasi middleware AuthorizeUser.php dengan kode berikut

```
1  <?php
2  namespace App\Http\Middleware;
3
4  use Closure;
5  use Illuminate\Http\Request;
6  use Symfony\Component\HttpFoundation\Response;
7
8  class AuthorizeUser
9  {
10     /**
11      * Handle an incoming request.
12      *
13      * @param \Closure(\Illuminate\Http\Request): (\Symfony\Component\HttpFoundation\Response) $next
14      */
15     public function handle(Request $request, Closure $next, ... $roles): Response
16     {
17         $user_role = $request->user()->getRole(); // ambil data level_kode dari user yg login
18         if(in_array($user_role, $roles)){ // cek apakah level_kode user ada di dalam array roles
19             return $next($request); // jika ada, maka lanjutkan request
20         }
21         // jika tidak punya role, maka tampilkan error 403
22         abort(403, 'Forbidden. Kamu tidak punya akses ke halaman ini');
23     }
24 }
```

Hasil :

```
app > Http > Middleware > AuthorizeUser.php > PHP > AuthorizeUser > handle()
1  <?php
2
3  namespace App\Http\Middleware;
4
5  use Closure;
6  use Illuminate\Http\Request;
7  use Symfony\Component\HttpFoundation\Response;
8
9  1 reference | 0 implementations
10 class AuthorizeUser
11 {
12     /**
13      * Handle an incoming request.
14      *
15      * @param \Closure(\Illuminate\Http\Request): (\Symfony\Component\HttpFoundation\Response) $next
16      */
17     0 references | 0 overrides
18     public function handle(Request $request, Closure $next, ...$roles): Response
19     {
20         $user_role = $request->user()->getRole(); // ambil data level_kode dari user yg login
21         if(in_array($user_role, $roles)){ // cek apakah level_kode user ada di dalam array roles
22             return $next($request); // jika ada, maka lanjutkan request
23         }
24         // jika tidak punya role, maka tampilkan error 403
25         abort(code: 403, message: 'Forbidden. Kamu tidak punya akses ke halaman ini');
26     }
27 }
```

3. Setelah itu tinggal kita perbaiki `route/web.php` sesuaikan dengan role/level yang diinginkan. Contoh

```
// artinya semua route di dalam group ini harus punya role ADM (Administrator) dan MNG (Manager)
Route::middleware(['authorize:ADM,MNG'])->group(function(){
    Route::get('/barang',[BarangController::class,'index']);
    Route::post('/barang/list',[BarangController::class,'list']);
    Route::get('/barang/create_ajax',[BarangController::class,'create_ajax']); // ajax form create
    Route::post('/barang_ajax',[BarangController::class,'store_ajax']); // ajax store
    Route::get('/barang/{id}/edit_ajax',[BarangController::class,'edit_ajax']); // ajax form edit
    Route::put('/barang/{id}/update_ajax',[BarangController::class,'update_ajax']); // ajax update
    Route::get('/barang/{id}/delete_ajax',[BarangController::class,'confirm_ajax']); // ajax form confirm
    Route::delete('/barang/{id}/delete_ajax',[BarangController::class,'delete_ajax']); // ajax delete
});
```



Hasil :

```
23 ✓ Route::group(['prefix' => 'user'], function (): void {  
24 ✓     Route::middleware(['authorize:ADM'])->group(function(): void {  
25         Route::get('/', [UserController::class, 'index']); // menampilkan halaman awal user  
  
43 Route::group(['prefix' => 'level'], function (): void {  
44     Route::middleware(['authorize:ADM'])->group(function (): void {  
45         Route::get('/', [LevelController::class, 'index']); // menampilkan halaman awal level  
  
63 Route::group(['prefix' => 'kategori'], function (): void {  
64     Route::middleware(['authorize:ADM,MNG,STF'])->group(function (): void {  
65         Route::get('/', [KategoriController::class, 'index']); // menampilkan halaman awal kategori  
  
83 Route::group(['prefix' => 'supplier'], function (): void {  
84     Route::middleware(['authorize:ADM,MNG,STF'])->group(function (): void {  
85         Route::get('/', [SupplierController::class, 'index']); // menampilkan halaman awal supplier  
  
103 Route::group(['prefix' => 'barang'], function (): void {  
104     Route::middleware(['authorize:ADM,MNG,STF'])->group(function (): void {  
105         Route::get('/', [BarangController::class, 'index']); // menampilkan halaman awal barang
```

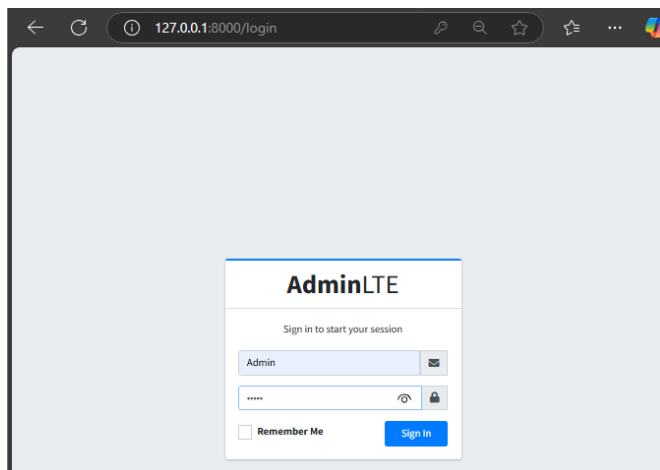
4. Sekarang kita sudah bisa memberikan hak akses menu/route ke beberapa level user

Tugas 3 – Implementasi Multi-Level Authorization :

1. Silahkan implementasikan multi-level authorization pada project kalian masing-masing

Hasil :

a. Admin





KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
<http://www.polinema.ac.id>

PWL - Starter Code

Search

Dashboard

Data Pengguna

Level User

Data User

Data Barang

Kategori Barang

Data Barang

Data Supplier

Data Transaksi

Stok Barang

Transaksi Penjualan

Lainnya

Logout

Home Contact

Daftar Level

Home / Level

Daftar Level yang terdaftar dalam sistem

Tambah Tambah Aja

Show 10 entries Search:

ID	Kode Level	Nama Level	Aksi
1	ADM	Administrator	Detail Edit Hapus
2	MNG	Manager	Detail Edit Hapus
3	STF	Staff/Kasir	Detail Edit Hapus
4	CUS2	Customer	Detail Edit Hapus

Showing 1 to 4 of 4 entries

Previous 1 Next

Copyright © 2014-2021 AdminLTE.io. All rights reserved.

Version 3.2.0

PWL - Starter Code

Search

Dashboard

Data Pengguna

Level User

Data User

Data Barang

Kategori Barang

Data Barang

Data Supplier

Data Transaksi

Stok Barang

Transaksi Penjualan

Lainnya

Logout

Home Contact

Daftar User

Home / User

Daftar user yang terdaftar dalam sistem

Tambah Tambah Aja

Filter: - Semua -

Level Pengguna

Show 10 entries Search:

ID	Username	Nama	Level Pengguna	Aksi
1	Admin	Administrator	Administrator	Detail Edit Hapus
2	Manager	Manager	Manager	Detail Edit Hapus
3	Staff	Staff/Kasir	Staff/Kasir	Detail Edit Hapus
4	customer-1	Manager 1	Customer	Detail Edit Hapus
5	manager_dua	Manager 2	Manager	Detail Edit Hapus
6	manager22	Manager Dua Dua	Manager	Detail Edit Hapus
7	manager33	Manager Tiga Tiga	Manager	Detail Edit Hapus
8	manager56	Manager 55	Manager	Detail Edit Hapus
9	manager12	Manager 11	Manager	Detail Edit Hapus
10	manager11	Manager 11	Manager	Detail Edit Hapus

Showing 1 to 10 of 11 entries

Previous 1 2 Next

Copyright © 2014-2021 AdminLTE.io. All rights reserved.

Version 3.2.0



Daftar Kategori yang terdaftar dalam sistem

ID	Kode Kategori	Kategori Nama	Aksi
1	KAT001	Makanan	Detail Edit Hapus
2	KAT002	MakeUp	Detail Edit Hapus
3	KAT003	Aksesoris	Detail Edit Hapus
4	KAT004	Pewangi	Detail Edit Hapus
5	KAT005	Alat Mandi	Detail Edit Hapus
6	CML	Camilan	Detail Edit Hapus
7	SDL	Sendal	Detail Edit Hapus

Showing 1 to 7 of 7 entries

Copyright © 2014-2021 AdminLTE.io. All rights reserved. Version 3.2.0

Daftar barang yang terdaftar dalam sistem

ID	Kode Barang	Nama Barang	Kategori	Harga Beli	Harga Jual	Aksi
1	BRG001	Kulkas	Makanan	500000	700000	Detail Edit Hapus
2	BRG002	Chopper	Makanan	300000	400000	Detail Edit Hapus
3	BRG003	Jaket	MakeUp	50000	100000	Detail Edit Hapus
4	BRG004	Maskara	MakeUp	100000	200000	Detail Edit Hapus
5	BRG005	Pensil	Aksesoris	2000	5000	Detail Edit Hapus
6	BRG006	Buku Tulis	Aksesoris	5000	10000	Detail Edit Hapus
7	BRG007	Kemeja	Pewangi	1000	2000	Detail Edit Hapus
8	BRG008	Neri	Pewangi	5000	10000	Detail Edit Hapus
9	BRG009	Jepit Rambut	Alat Mandi	3000	5000	Detail Edit Hapus
10	BRG010	Sabun	Alat Mandi	5000	8000	Detail Edit Hapus

Showing 1 to 10 of 11 entries

Copyright © 2014-2021 AdminLTE.io. All rights reserved. Version 3.2.0

Daftar supplier yang terdaftar dalam sistem

ID	Kode Supplier	Supplier Nama	Alamat Supplier	Aksi
1	SDY001	PratamaStore	Tulungagung	Detail Edit Hapus
2	SDY002	AwarPulih	Malang	Detail Edit Hapus
3	SDY003	Mearky	Jakarta	Detail Edit Hapus
4	SDY004	capaciti	Trompsguk	Detail Edit Hapus

Showing 1 to 4 of 4 entries

Copyright © 2014-2021 AdminLTE.io. All rights reserved. Version 3.2.0

b. Manager



← ↻ 127.0.0.1:8000/login

AdminLTE

Sign in to start your session

☐ Remember Me

Sign in

← ↻ 127.0.0.1:8000/kategori

PWL - Starter Code

Search

Dashboard

Data Pengguna

Level User

Data User

Data Barang

Kategori Barang

Data Barang

Data Supplier

Data Transaksi

Stok Barang

Transaksi Penjualan

Lainnya

Logout

Daftar Kategori

Daftar kategori yang terdaftar dalam sistem

Show 1 to 7 entries

ID	Kode Kategori	Kategori Nama	Aksi
1	KAT001	Makanan	Detail Edit Hapus
2	KAT002	Makelup	Detail Edit Hapus
3	KAT003	Akesoris	Detail Edit Hapus
4	KAT004	Pewangi	Detail Edit Hapus
5	KAT005	Alat Mandi	Detail Edit Hapus
6	CHL	Camilan	Detail Edit Hapus
7	SDL	Sandal	Detail Edit Hapus

Showing 1 to 7 of 7 entries

Previous 1 Next

← ↻ 127.0.0.1:8000/barang

PWL - Starter Code

Search

Dashboard

Data Pengguna

Level User

Data User

Data Barang

Data Barang

Data Barang

Data Supplier

Data Transaksi

Stok Barang

Transaksi Penjualan

Lainnya

Logout

Daftar Barang

Daftar barang yang terdaftar dalam sistem

Filter: Semua

Show 1 to 10 entries

ID	Kode Barang	Nama Barang	Kategori	Harga Beli	Harga Jual	Aksi
1	BRG001	Kulkas	Makanan	500000	700000	Detail Edit Hapus
2	BRG002	Chopper	Makanan	300000	400000	Detail Edit Hapus
3	BRG003	Jaket	Makelup	50000	100000	Detail Edit Hapus
4	BRG004	Maskara	Makelup	100000	200000	Detail Edit Hapus
5	BRG005	Pensil	Akesoris	2000	5000	Detail Edit Hapus
6	BRG006	Buku Tulis	Akesoris	5000	10000	Detail Edit Hapus
7	BRG007	Kemupak	Pewangi	3000	2000	Detail Edit Hapus
8	BRG008	Nasi	Pewangi	5000	10000	Detail Edit Hapus
9	BRG009	Jepit Rambut	Alat Mandi	3000	5000	Detail Edit Hapus
10	BRG010	Sabun	Alat Mandi	5000	8000	Detail Edit Hapus

← ↻ 127.0.0.1:8000/supplier

PWL - Starter Code

Search

Dashboard

Data Pengguna

Level User

Data User

Data Barang

Data Barang

Data Supplier

Data Transaksi

Stok Barang

Transaksi Penjualan

Lainnya

Logout

Daftar Supplier

Daftar supplier yang terdaftar dalam sistem

Show 1 to 4 entries

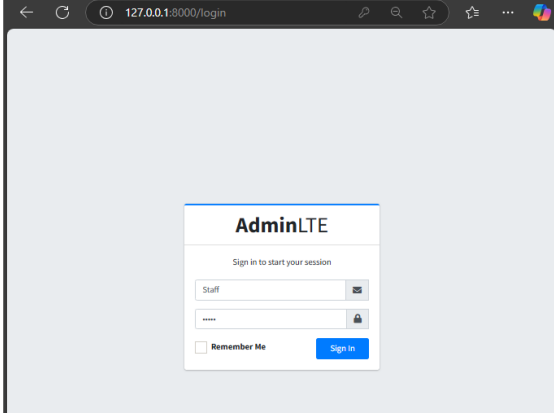
ID	Kode Supplier	Supplier Nama	Alamat Supplier	Aksi
1	BDH001	PratamaStore	Tulungagung	Detail Edit Hapus
2	BDH002	AzanPutih	Malang	Detail Edit Hapus
3	BDH003	Mauricy	Jakarta	Detail Edit Hapus
4	BDH004	capacupi	Trenggalek	Detail Edit Hapus

Showing 1 to 4 of 4 entries

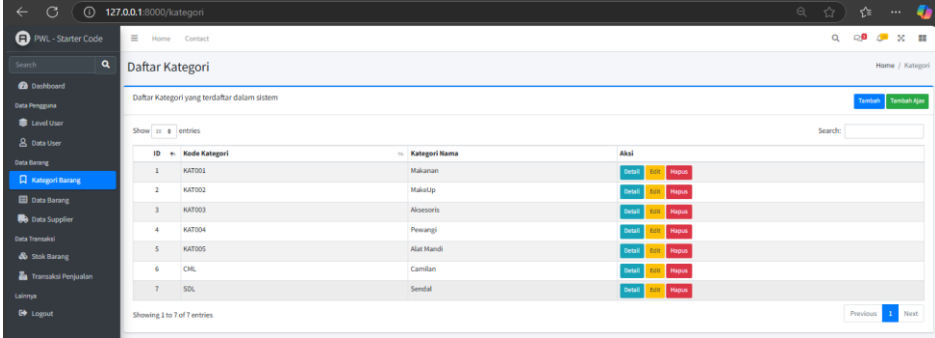
Previous 1 Next



c. Staff

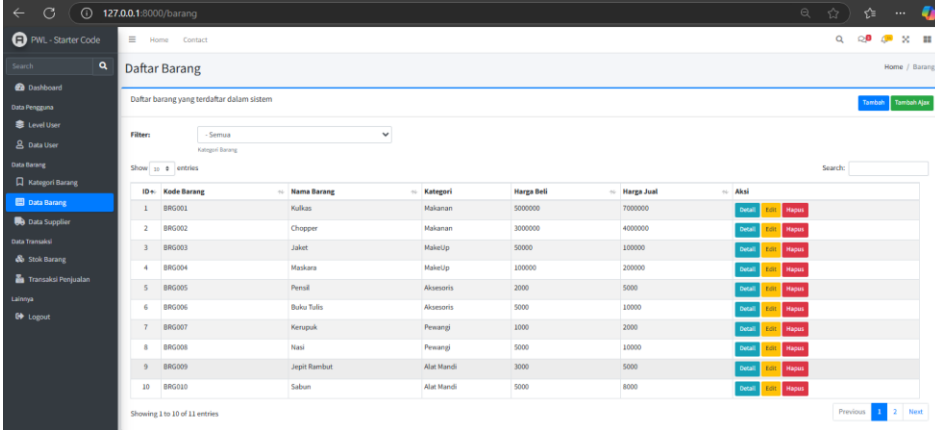


The first screenshot shows the AdminLTE login page. It has a title "AdminLTE" and a subtitle "Sign in to start your session". There is a "Staff" label, a password field, a "Remember Me" checkbox, and a "Sign In" button.



The second screenshot shows the "Daftar Kategori" (Category List) page. It displays a table of categories with columns: ID, Kode Kategori, Kategori Nama, and Aksi. The table contains 7 entries. The "Aksi" column has buttons for "Detail", "Edit", and "Hapus".

ID	Kode Kategori	Kategori Nama	Aksi
1	KAT001	Makanan	Detail Edit Hapus
2	KAT002	MakeUp	Detail Edit Hapus
3	KAT003	Aksesoris	Detail Edit Hapus
4	KAT004	Pewangi	Detail Edit Hapus
5	KAT005	Alat Mandi	Detail Edit Hapus
6	CML	Camilan	Detail Edit Hapus
7	SDL	Sendal	Detail Edit Hapus



The third screenshot shows the "Daftar Barang" (Goods List) page. It displays a table of goods with columns: ID, Kode Barang, Nama Barang, Kategori, Harga Beli, Harga Jual, and Aksi. The table contains 10 entries. The "Aksi" column has buttons for "Detail", "Edit", and "Hapus".

ID	Kode Barang	Nama Barang	Kategori	Harga Beli	Harga Jual	Aksi
1	BRG001	Kulkas	Makanan	500000	700000	Detail Edit Hapus
2	BRG002	Chopper	Makanan	300000	400000	Detail Edit Hapus
3	BRG003	Jaket	MakeUp	50000	100000	Detail Edit Hapus
4	BRG004	Maskara	MakeUp	100000	200000	Detail Edit Hapus
5	BRG005	Pemil	Aksesoris	2000	5000	Detail Edit Hapus
6	BRG006	Buku Tulis	Aksesoris	5000	10000	Detail Edit Hapus
7	BRG007	Kenguk	Pewangi	1000	2000	Detail Edit Hapus
8	BRG008	Nasi	Pewangi	5000	10000	Detail Edit Hapus
9	BRG009	Jepit Rambut	Alat Mandi	3000	5000	Detail Edit Hapus
10	BRG010	Sabun	Alat Mandi	5000	8000	Detail Edit Hapus

2. Amati dan jelaskan tiap tahapan yang kalian kerjakan, dan jabarkan dalam laporan
3. Implementasikan multi-level authorization untuk semua Level/Jenis User dan Menu yang sesuai dengan Level/Jenis User

Hasil :



- User

```
23 Route::group(['prefix' => 'user'], function () { void {
24   Route::middleware(['authorize:ADM'])->group(function () { void {
25     Route::get('/', [UserController::class, 'index']); // menampilkan halaman awal user
26     Route::post('/list', [UserController::class, 'list']); // menampilkan data user dalam bentuk json untuk datatables
27     Route::get('/create', [UserController::class, 'create']); // menampilkan halaman form tambah user
28     Route::post('/', [UserController::class, 'store']); // menyimpan data user baru
29     Route::get('/create_ajax', [UserController::class, 'create_ajax']); // menampilkan halaman form tambah user ajax
30     Route::post('/ajax', [UserController::class, 'store_ajax']); // menyimpan data user baru ajax
31     Route::get('/{id}', [UserController::class, 'show']); // menampilkan detail user
32     Route::get('/{id}/edit', [UserController::class, 'edit']); // menampilkan halaman form edit user
33     Route::put('/{id}', [UserController::class, 'update']); // menyimpan perubahan data user
34     Route::get('/{id}/show_ajax', [UserController::class, 'show_ajax']); // menampilkan halaman detail user ajax
35     Route::get('/{id}/edit_ajax', [UserController::class, 'edit_ajax']); // menampilkan halaman form edit user ajax
36     Route::put('/{id}/update_ajax', [UserController::class, 'update_ajax']); // menyimpan perubahan data user ajax
37     Route::get('/{id}/delete_ajax', [UserController::class, 'confirm_ajax']); // menampilkan halaman konfirmasi delete user ajax
38     Route::delete('/{id}/delete_ajax', [UserController::class, 'delete_ajax']); // menghapus data user ajax
39     Route::delete('/{id}', [UserController::class, 'destroy']); // menghapus data user
40   });
41 });
```

- Level

```
43 Route::group(['prefix' => 'level'], function () { void {
44   Route::middleware(['authorize:ADM'])->group(function () { void {
45     Route::get('/', [LevelController::class, 'index']); // menampilkan halaman awal level
46     Route::post('/list', [LevelController::class, 'list']); // menampilkan data level dalam bentuk json untuk datatables
47     Route::get('/create', [LevelController::class, 'create']); // menampilkan halaman form tambah level
48     Route::post('/', [LevelController::class, 'store']); // menyimpan data level baru
49     Route::get('/create_ajax', [LevelController::class, 'create_ajax']); // menampilkan halaman form tambah level ajax
50     Route::post('/ajax', [LevelController::class, 'store_ajax']); // menyimpan data level baru ajax
51     Route::get('/{id}', [LevelController::class, 'show']); // menampilkan detail level
52     Route::get('/{id}/edit', [LevelController::class, 'edit']); // menampilkan halaman form edit level
53     Route::put('/{id}', [LevelController::class, 'update']); // menyimpan perubahan data level
54     Route::get('/{id}/show_ajax', [LevelController::class, 'show_ajax']); // menampilkan halaman detail level ajax
55     Route::get('/{id}/edit_ajax', [LevelController::class, 'edit_ajax']); // menampilkan halaman form edit level ajax
56     Route::put('/{id}/update_ajax', [LevelController::class, 'update_ajax']); // menyimpan perubahan data level ajax
57     Route::get('/{id}/delete_ajax', [LevelController::class, 'confirm_ajax']); // menampilkan halaman konfirmasi delete level ajax
58     Route::delete('/{id}/delete_ajax', [LevelController::class, 'delete_ajax']); // menghapus data level ajax
59     Route::delete('/{id}', [LevelController::class, 'destroy']); // menghapus data level
60   });
61 });
```

- Kategori

```
63 Route::group(['prefix' => 'kategori'], function () { void {
64   Route::middleware(['authorize:ADM,MNG,STF'])->group(function () { void {
65     Route::get('/', [KategoriController::class, 'index']); // menampilkan halaman awal kategori
66     Route::post('/list', [KategoriController::class, 'list']); // menampilkan data kategori dalam bentuk json untuk datatables
67     Route::get('/create', [KategoriController::class, 'create']); // menampilkan halaman form tambah kategori
68     Route::post('/', [KategoriController::class, 'store']); // menyimpan data kategori baru
69     Route::get('/create_ajax', [KategoriController::class, 'create_ajax']); // menampilkan halaman form tambah kategori ajax
70     Route::post('/ajax', [KategoriController::class, 'store_ajax']); // menyimpan data kategori baru ajax
71     Route::get('/{id}', [KategoriController::class, 'show']); // menampilkan detail kategori
72     Route::get('/{id}/edit', [KategoriController::class, 'edit']); // menampilkan halaman form edit kategori
73     Route::put('/{id}', [KategoriController::class, 'update']); // menyimpan perubahan data kategori
74     Route::get('/{id}/show_ajax', [KategoriController::class, 'show_ajax']); // menampilkan halaman detail kategori ajax
75     Route::get('/{id}/edit_ajax', [KategoriController::class, 'edit_ajax']); // menampilkan halaman form edit kategori ajax
76     Route::put('/{id}/update_ajax', [KategoriController::class, 'update_ajax']); // menyimpan perubahan data kategori ajax
77     Route::get('/{id}/delete_ajax', [KategoriController::class, 'confirm_ajax']); // menampilkan halaman konfirmasi delete kategori ajax
78     Route::delete('/{id}/delete_ajax', [KategoriController::class, 'delete_ajax']); // menghapus data kategori ajax
79     Route::delete('/{id}', [KategoriController::class, 'destroy']); // menghapus data kategori
80   });
81 });
```

- Supplier

```
83 Route::group(['prefix' => 'supplier'], function () { void {
84   Route::middleware(['authorize:ADM,MNG'])->group(function () { void {
85     Route::get('/', [SupplierController::class, 'index']); // menampilkan halaman awal supplier
86     Route::post('/list', [SupplierController::class, 'list']); // menampilkan data supplier dalam bentuk json untuk datatables
87     Route::get('/create', [SupplierController::class, 'create']); // menampilkan halaman form tambah supplier
88     Route::post('/', [SupplierController::class, 'store']); // menyimpan data supplier baru
89     Route::get('/create_ajax', [SupplierController::class, 'create_ajax']); // menampilkan halaman form tambah supplier ajax
90     Route::post('/ajax', [SupplierController::class, 'store_ajax']); // menyimpan data supplier baru ajax
91     Route::get('/{id}', [SupplierController::class, 'show']); // menampilkan detail supplier
92     Route::get('/{id}/edit', [SupplierController::class, 'edit']); // menampilkan halaman form edit supplier
93     Route::put('/{id}', [SupplierController::class, 'update']); // menyimpan perubahan data supplier
94     Route::get('/{id}/show_ajax', [SupplierController::class, 'show_ajax']); // menampilkan halaman detail supplier ajax
95     Route::get('/{id}/edit_ajax', [SupplierController::class, 'edit_ajax']); // menampilkan halaman form edit supplier ajax
96     Route::put('/{id}/update_ajax', [SupplierController::class, 'update_ajax']); // menyimpan perubahan data supplier ajax
97     Route::get('/{id}/delete_ajax', [SupplierController::class, 'confirm_ajax']); // menampilkan halaman konfirmasi delete supplier ajax
98     Route::delete('/{id}/delete_ajax', [SupplierController::class, 'delete_ajax']); // menghapus data supplier ajax
99     Route::delete('/{id}', [SupplierController::class, 'destroy']); // menghapus data supplier
100   });
101 });
```



- Barang

```
103 Route::group(['prefix' => 'barang'], function () { void {
104     Route::middleware(['authorize:ADM,MWG,STF'])->group(function () { void {
105         Route::get('/', [BarangController::class, 'index']); // menampilkan halaman awal barang
106         Route::post('/list', [BarangController::class, 'list']); // menampilkan data barang dalam bentuk json untuk datatables
107         Route::get('/create', [BarangController::class, 'create']); // menampilkan halaman form tambah barang
108         Route::post('/', [BarangController::class, 'store']); // menyimpan data barang baru
109         Route::get('/create_ajax', [BarangController::class, 'create_ajax']); // menampilkan halaman form tambah barang ajax
110         Route::post('/ajax', [BarangController::class, 'store_ajax']); // menyimpan data barang baru ajax
111         Route::get('/{id}', [BarangController::class, 'show']); // menampilkan detail barang
112         Route::get('/{id}/edit', [BarangController::class, 'edit']); // menampilkan halaman form edit barang
113         Route::put('/{id}', [BarangController::class, 'update']); // menyimpan perubahan data barang
114         Route::get('/{id}/show_ajax', [BarangController::class, 'show_ajax']); // menampilkan halaman detail barang ajax
115         Route::get('/{id}/edit_ajax', [BarangController::class, 'edit_ajax']); // menampilkan halaman form edit barang ajax
116         Route::put('/{id}/update_ajax', [BarangController::class, 'update_ajax']); // menyimpan perubahan data barang ajax
117         Route::get('/{id}/delete_ajax', [BarangController::class, 'confirm_ajax']); // menampilkan halaman konfirmasi delete barang ajax
118         Route::delete('/{id}/delete_ajax', [BarangController::class, 'delete_ajax']); // menghapus data barang ajax
119         Route::delete('/{id}', [BarangController::class, 'destroy']); // menghapus data barang
120     });
121 });
```

4. Submit kode untuk impementasi Authorization pada repository github kalian.

Tugas 4 – Implementasi Form Registrasi :

1. Silahkan implementasikan form untuk registrasi user.

Hasil :

- AuthController.php

```
app > Http > Controllers > AuthController.php > PHP > AuthController > postregister()
10 class AuthController extends Controller
48
49 public function register(): Factory|View
50 {
51     $level = LevelModel::all(); // Ambil semua level dari database
52     return view('auth.register', data: compact('level'));
53 }
54
55 public function postregister(Request $request): mixed|RedirectResponse
56 {
57     $request->validate([
58         'username' => 'required',
59         'nama' => 'required',
60         'password' => 'required|min:5',
61         'level_id' => 'required|exists:m_level,level_id' // Validasi level harus ada di tabel m_level
62     ]);
63
64     $user = new UserModel();
65     $user->username = $request->username;
66     $user->nama = $request->nama;
67     $user->password = bcrypt(value: $request->password);
68     $user->level_id = $request->level_id;
69     $user->save();
70
71     return redirect(to: 'login')->with('success', 'Registrasi berhasil! Silakan login.');
```



- **UserController.php**

```
app > Http > Controllers > UserController.php > PHP > UserController > delete_ajax()
13 class UserController extends Controller
14 {
15     1 reference | 0 overrides
16     public function delete_ajax(Request $request, $id): JsonResponse|mixed|Redirector|RedirectRes...
17     {
18         // cek apakah request dari ajax
19         if ($request->ajax() || $request->wantsJson()) {
20             $user = UserModel::find($id);
21             if ($user) {
22                 try {
23                     // Cek apakah user yang dihapus adalah user yang sedang login
24                     if ($user->username == Auth::user()->username) {
25                         $logout = true;
26                     }
27                     // Menghapus data user berdasarkan ID
28                     UserModel::destroy($id);
29                     if ($logout) {
30                         return response()->json([
31                             'status' => true,
32                             'logout' => true,
33                             'message' => 'Data user berhasil dihapus'
34                         ]);
35                     }
36                     return response()->json([
37                         'status' => true,
38                         'message' => 'Data user berhasil dihapus'
39                     ]);
40                 } catch (\Illuminate\Database\QueryException $e) {
41                     // Jika terjadi error ketika menghapus data,
42                     // redirect kembali ke halaman dengan pesan error
43                     return response()->json([
44                         'status' => false,
45                         'message' => 'Data user gagal dihapus karena masih terdapat tabel lain yang terkait dengan data ini'
46                     ]);
47                 }
48             }
49         }
50     }
51 }
```

- **Login.blade.php**

```
resources > views > auth > login.blade.php > html
2 <html lang="en">
19 <body class="hold-transition login-page">
20 <div class="login-box">
21 <div class="card card-outline card-primary">
22 <div class="card-body">
23     @if (session(key: 'success'))
24         <div class="alert alert-success text-center">{{ session(key: 'success') }}</div>
25     @endif
26     @if (session(key: 'error'))
27         <div class="alert alert-danger text-center">{{ session(key: 'error') }}</div>
28     @endif
29     <p class="login-box-msg">Sign in to start your session</p>
30     <form action="{{ url(path: 'login') }}" method="post" id="form-login">
31         @csrf
32     </form>
33 </div>
34 </div>
68 <div class="text-center mt-4">
69     <p>Don't have an account? <a href="{{ url(path: 'register') }}">Register</a></p>
70 </div>
71 </form>
```

- **Register.blade.php**



```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <meta name="csrf-token" content="{{ csrf_token() }}">
8   <title>Registrasi Pengguna</title>
9
10  <link rel="stylesheet"
11    href="https://fonts.googleapis.com/css?family=Source+Sans+Pro:300,400,400i,700&display=fallback">
12  <link rel="stylesheet" href="{{ asset('adminlte/plugins/fontawesome-free/css/all.min.css') }}">
13  <link rel="stylesheet" href="{{ asset('adminlte/plugins/icheck-bootstrap/icheck-bootstrap.min.css') }}">
14  <link rel="stylesheet" href="{{ asset('adminlte/plugins/sweetalert2-theme-bootstrap-4/bootstrap-4.min.css') }}">
15  <link rel="stylesheet" href="{{ asset('adminlte/dist/css/adminlte.min.css') }}">
16 </head>
17
18 <body class="hold-transition login-page">
19   <div class="login-box">
20     <div class="card card-outline card-primary">
21       <div class="card-header text-center">
22         <a href="{{ url('/') }}" class="text-decoration: none; color: #007bff;">
23           <b>Admin</b> / <b>LTE</b> / <b>A</b>
24         </div>
25       <div class="card-body">
26         <p class="text-center">Daftar Pengguna Baru</p>
27         <form action="{{ url('register') }}" method="POST" id="form-register">
28           <div class="form-group">
29             <label>Pilih Role</label>
30             <select name="level_id" id="level_id" class="form-control" required>
31               <option value="">Pilih Level --</option>
32               <foreach $level as $l>
33                 <option value="{{ $l->level_id }}">{{ $l->level_nama }}</option>
34               </foreach>
35             </select>
36             <small id="error-level" class="error-text form-text text-danger"></small>
37           </div>
38           <div class="form-group">
39             <label>Username</label>
40             <input type="text" name="username" id="username" class="form-control" required minlength="4"
41               maxlength="20" placeholder="Masukkan Username">
42             <small id="error-username" class="error-text form-text text-danger"></small>
43           </div>
44           <div class="form-group">
45             <label>Nama</label>
46             <input type="text" name="nama" id="nama" class="form-control" required maxlength="100" placeholder="Masukkan Nama">
47             <small id="error-nama" class="error-text form-text text-danger"></small>
48           </div>
49           <div class="form-group">
50             <label>Password</label>
51             <input type="password" name="password" id="password" class="form-control" required minlength="5"
52               maxlength="20" placeholder="Masukkan Password">
53             <small id="error-password" class="error-text form-text text-danger"></small>
54           </div>
55           <div class="row justify-content-end">
56             <div class="col-auto">
57               <button type="submit" class="btn btn-primary btn-block">Sign Up</button>
58             </div>
59             <div class="text-center mt-3">
60               <p>Sudah punya akun? <a href="{{ url('login') }}">Login</a></p>
61             </div>
62           </div>
63         </form>
64       </div>
65     </div>
66   </div>
67
68   <script src="{{ asset('adminlte/plugins/jquery/jquery.min.js') }}"></script>
69   <script src="{{ asset('adminlte/plugins/bootstrap/js/bootstrap.bundle.min.js') }}"></script>
70   <script src="{{ asset('adminlte/plugins/jquery-validation/jquery.validate.min.js') }}"></script>
71   <script src="{{ asset('adminlte/plugins/jquery-validation/additional-methods.min.js') }}"></script>
72   <script src="{{ asset('adminlte/plugins/sweetalert2/sweetalert2.min.js') }}"></script>
73   <script src="{{ asset('adminlte/dist/js/adminlte.min.js') }}"></script>
74
75   <script>
76     $(document).ready(function () {
77       $("#form-register").validate({
78         rules: {
79           username: { required: true, minlength: 4, maxlength: 20 },
80           nama: { required: true, maxlength: 100 },
81           password: { required: true, minlength: 5, maxlength: 20 },
82           password_confirmation: { equalTo: "password" },
83           level_id: { required: true, number: true }
84         },
85         submitHandler: function (form) {
86           $.ajax({
87             headers: {
88               'X-CSRF-TOKEN': $('meta[name="csrf-token"]').attr('content')
89             },
90             url: form.action,
91             type: form.method,
92             data: $(form).serialize(),
93             success: function (response) {
94               if (response.status) {
95                 Swal.fire({
96                   icon: 'success',
97                   title: 'Registrasi Berhasil',
98                   text: response.message,
99                 }).then(() => {
100                   if (response.redirect) {
101                     window.location = response.redirect;
102                   }
103                 });
104               } else {
105                 $.each(response.errors, function (key, val) {
106                   $('#error-' + key).text(val[0]);
107                 });
108                 Swal.fire({
109                   icon: 'error',
110                   title: 'Terjadi Kesalahan',
111                   text: response.message
112                 });
113               }
114             }
115           });
116           return false;
117         },
118         errorElement: 'span',
119         errorPlacement: function (error, element) {
120           error.addClass('invalid-feedback');
121           element.closest('.input-group').append(error);
122         },
123         highlight: function (element) {
124           $(element).addClass('is-invalid');
125         },
126         unhighlight: function (element) {
127           $(element).removeClass('is-invalid');
128         }
129       });
130     });
131   </script>
132 </body>
133 </html>
```



- **Confirm_ajax.blade.php**

```
resources > views > user > confirm_ajax.blade.php > ...
44 <script>
45 $(document).ready(function() {
48     submitHandler: function(form) {
53         success: function(response) {
58             text: response.message
59         });
60     });
61     // Cek apakah user yang dihapus adalah user yang sedang login
62     if (response.logout) {
63         window.location.href = "/logout"; // Redirect ke halaman logout
64     } else {
65         dataUser.ajax.reload(); // Reload tabel jika bukan user yang sedang login
66     }
67 }else{
```

- **Web.php**

```
14 Route::get('login', [AuthController::class, 'login'])->name('login');
15 Route::post('login', [AuthController::class, 'postlogin']);
16 Route::get('logout', [AuthController::class, 'logout'])->middleware('auth');
17 Route::get('register', [AuthController::class, 'register']);
18 Route::post('register', [AuthController::class, 'PostRegister']);
19
```

2. Screenshot hasil yang kalian kerjakan

Hasil :

AdminLTE

Daftar Pengguna Baru

Pilih Role

Staff/Kasir

Username

Staffclaudya

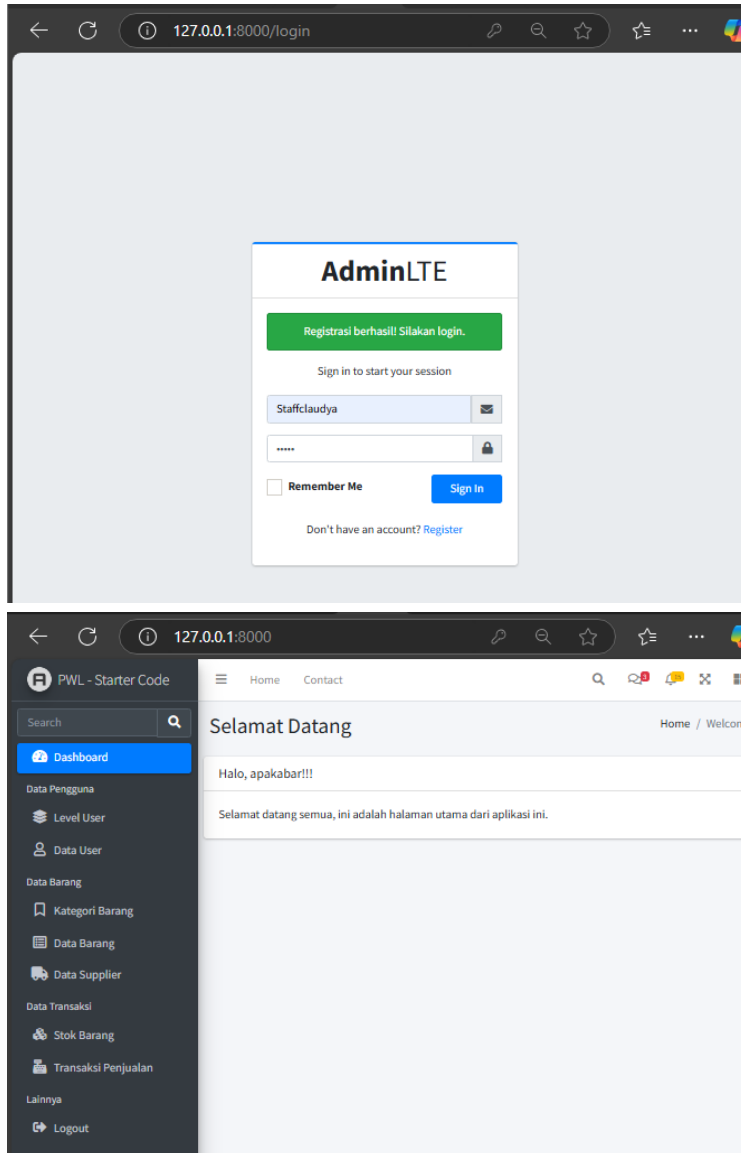
Nama

Claudya

Password

Sign Up

Sudah punya akun? [Login](#)



3. Commit dan push hasil tugas kalian ke masing-masing repo github kalian

**** Sekian, dan selamat belajar ****